

# QUANTUM COMPUTING PROJECT

PRESENTATION

# CONTENT

---

Paper explanation

News

company

QUANTUM CIRCUIT OPTIMIZATION  
FOR THE FTQC ERA

DO WE HAVE TO START  
FROM SCRATCH?

## Noisy Intermediate-Scale Quantum NISQ

- Current hardware, but **heavily affected by noise**
- Optimization techniques reduce noise, but **only up to a certain point**
- Works for **simple applications**
- Main challenge is **noise from gate errors**
- Goal is **reducing gate count** to lower noise
- Limitation: cannot scale to industry problem

## Fault-Tolerant Quantum Computing FTQC

- Future hardware with **error correction**
- Error correction makes execution reliable but introduces **huge qubit overheads**
- Needed for **industrial-scale applications**
- Main challenge: **tens of thousands of qubits required** even for toy problems
- Goal is reducing **costly gates & qubit overhead** for scalability
- Limitation: overhead is unrealistic with today's hardware

# DO WE HAVE TO DEVELOP FTQC QUANTUM CIRCUIT OPTIMIZATION FROM SCRATCH?

---

- The results confirm that the optimization tools developed in the past years for NISQ circuits can improve FTQC hardware resource requirements. So that , FTQC quantum circuit optimization does not have to be developed from scratch. At the same time, the investigations provide indications of what optimization effects lead to stronger improvements for FTQC compared to others. Overall, providing these insights provides a foundation for the transfer of NISQ optimization techniques to the FTQC era, where different metrics are crucial for successful optimization

# WHY DO WE NEED TO SHIFT TO FTQC ERA?

---

- Quantum circuits can be executed by both simulators and current NISQ devices. Unfortunately, simulators are very limited by their number of simulatable qubits while the NISQ devices suffer from severe execution errors—getting more severe with an increasing number of qubits and gates. Therefore, to execute larger circuits, quantum computing needs to include error-correction to counteract this behavior and execute quantum algorithms in a fault-tolerant fashion.
- The no-cloning theorem states the impossibility of cloning quantum states. Thus, to account for the destruction of quantum states during a measurement, quantum error-correction techniques are more complex than classical error-correction techniques.



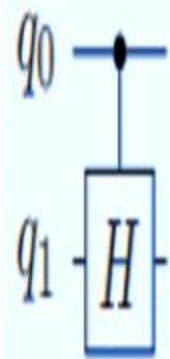
# QUANTUM CIRCUIT SYNTHESIS AND OPTIMIZATION

→THE SYNTHESIS PASS TRANSLATES THE SET OF ALL GATES OF A CIRCUIT TO A SEQUENCE OF GATES ACTUALLY EXECUTABLE ON THE CHOSEN DEVICE.

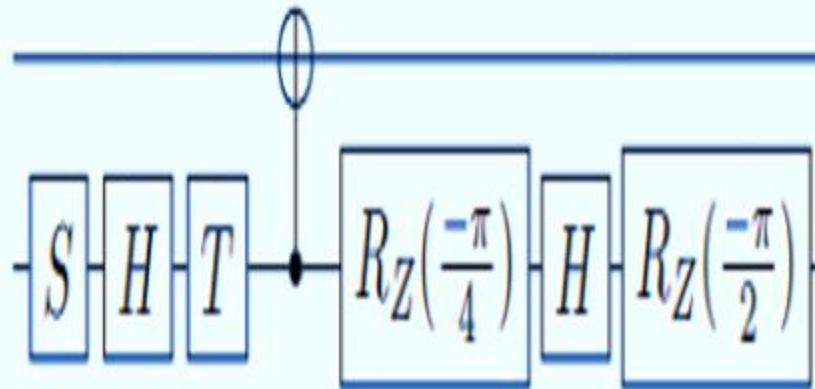
→OPTIMIZATION PASSES ARE USED TO REDUCE THIS GATE OVERHEAD AND MAKE SURE THAT ALL THE GATES ARE FROM THE NATIVE GATE SET OF THE DEVICE

# THE EFFECT OF SYNTHESIS AND OPTIMIZATION ON QUANTUM CIRCUIT

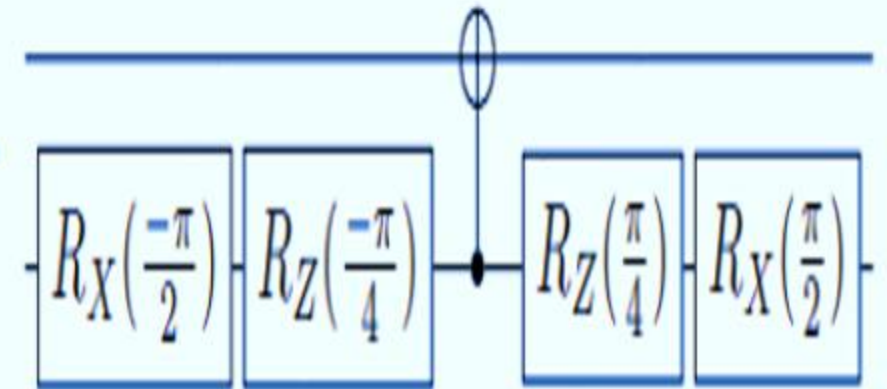
Quantum circuit



Synthesized circuit



Optimized circuit





# RELATIVE CHANGES IN PERCENT FOR THE CONSIDERED QISKIT OPTIMIZATION PASSES

|                  | Commutative-Cancellation |     |      | Optimize-1qGatesDecomposition |    |      | Optimize-1qGatesSimpleCommutation |    |      | Hoare-Optimizer |     |      | Template-Optimization |     |      |
|------------------|--------------------------|-----|------|-------------------------------|----|------|-----------------------------------|----|------|-----------------|-----|------|-----------------------|-----|------|
|                  | NISQ                     |     | FTQC | NISQ                          |    | FTQC | NISQ                              |    | FTQC | NISQ            |     | FTQC | NISQ                  |     | FTQC |
|                  | #G                       | #Q  | t    | #G                            | #Q | t    | #G                                | #Q | t    | #G              | #Q  | t    | #G                    | #Q  | t    |
| ae               | -11                      | -44 | -15  | -25                           | 0  | 0    | -25                               | 0  | 0    | -3              | 0   | 0    | 0                     | +11 | -11  |
| dj               | -9                       | 0   | 0    | -41                           | 0  | 0    | -41                               | 0  | 0    | -11             | 0   | 0    | 0                     | 0   | 0    |
| graphstate       | -39                      | 0   | 0    | -39                           | 0  | 0    | -31                               | 0  | 0    | -27             | 0   | 0    | 0                     | 0   | 0    |
| grover-noancilla | -2                       | 0   | 0    | -4                            | 0  | 0    | -4                                | 0  | 0    | -10             | -22 | -2   | 0                     | 0   | 0    |
| grover-v-chain   | -5                       | 0   | 0    | -8                            | 0  | 0    | -8                                | 0  | 0    | -4              | -41 | -1   | 0                     | 0   | 0    |
| qft              | -14                      | -40 | -23  | 0                             | 0  | 0    | 0                                 | 0  | 0    | -17             | -43 | -27  | 0                     | -24 | +34  |
| qftentangled     | -13                      | -40 | -23  | 0                             | 0  | 0    | 0                                 | 0  | 0    | 0               | 0   | 0    | 0                     | -24 | +34  |
| qpeexact         | -14                      | -38 | -28  | 0                             | 0  | -2   | 0                                 | 0  | -2   | -3              | +21 | -22  | 0                     | -3  | +2   |
| qpeinexact       | -14                      | -38 | -28  | 0                             | 0  | 0    | 0                                 | 0  | 0    | -3              | +32 | -30  | 0                     | -4  | +4   |
| qwalk-noancilla  | -2                       | +5  | -1   | -2                            | 0  | 0    | -2                                | 0  | 0    | -9              | +5  | -8   | 0                     | 0   | 0    |
| qwalk-v-chain    | -11                      | 0   | -4   | -14                           | +7 | -12  | -14                               | +7 | -12  | -31             | +15 | -45  | -2                    | 0   | 0    |

# RELATIVE CHANGES IN PERCENT FOR THE CONSIDERED QISKIT OPTIMIZATION PASSES

---

## *Findings:*

→optimizing Clifford gates improves NISQ more than FTQC in Deutsch-Jozsa (“dj”) , Amplitude Estimation (“ae”) .

→Reducing the Number of Rotation Gates Leads to Strong FTQC Benefits in the Quantum Fourier Transform (“qft”) or Quantum Phase Estimation (“qpe”) processed by Qiskit.

→Rearranging the Circuit Leads to Redistributions of FTQC Metrics:In the case of some optimization processes, the structure of the quantum circuit changes. So, some gates change their position by being commuted through the quantum circuit.

# RELATIVE CHANGES IN PERCENT FOR THE CONSIDERED QISKIT OPTIMIZATION PASSES

---

1. **Commutative-Cancellation:** Best for both NISQ & FTQC (big savings).
2. **1qGate Decomposition:** Only helps NISQ, no FTQC effect.
3. **1qGate Simple Commutation:** Purely NISQ, no FTQC benefit.
4. **Hoare-Optimizer:** Mixed in FTQC (can save or worsen resources).
5. **Template-Optimization:** Risky – may save qubits but hurt runtime in FTQ

# Problemsolver\_paper\_figures notebook

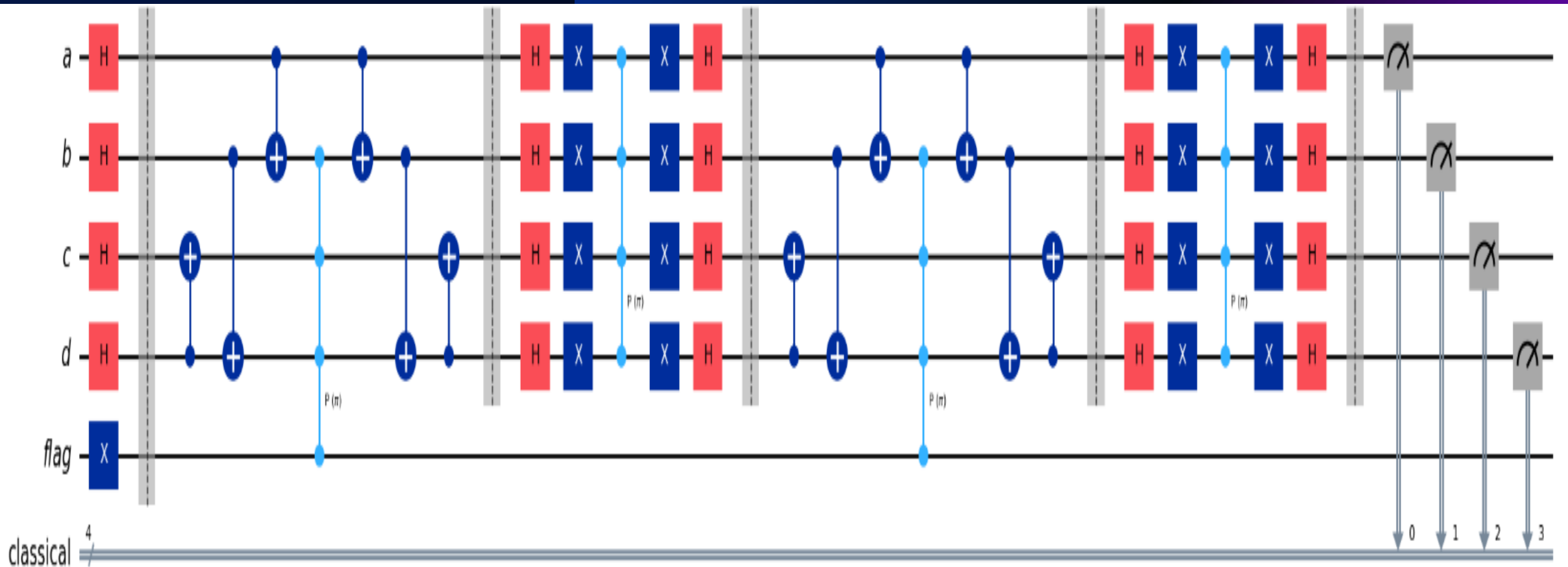
```
import numpy as np
from qiskit import ClassicalRegister, QuantumCircuit, QuantumRegister
from qiskit_algorithms import Grover

num_iterations = Grover.optimal_num_iterations(2, 4)
a = QuantumRegister(1, "a")
b = QuantumRegister(1, "b")
c = QuantumRegister(1, "c")
d = QuantumRegister(1, "d")
flag = QuantumRegister(1, "flag")
creg_c = ClassicalRegister(4, "classical")
circuit = QuantumCircuit(a, b, c, d, flag, creg_c)

circuit.h(a)
circuit.h(b)
circuit.h(c)
circuit.h(d)
circuit.x(flag)
circuit.barrier(a, b, c, d, flag)
for _i in range(2):
    circuit.cx(d, c)
    circuit.cx(b, d)
    circuit.cx(a, b)
    circuit.mcp(np.pi, [b, c, d], flag)
    circuit.cx(a, b)
    circuit.cx(b, d)
    circuit.cx(d, c)
    circuit.barrier(a, b, c, d)
    circuit.h(a)
    circuit.h(b)
    circuit.h(c)
    circuit.h(d)
```

```
circuit.h(c)
circuit.h(d)
circuit.x(d)
circuit.x(a)
circuit.x(b)
circuit.x(c)
circuit.mcp(np.pi, [a, b, c], d)
circuit.x(d)
circuit.x(c)
circuit.x(b)
circuit.x(a)
circuit.h(d)
circuit.h(c)
circuit.h(b)
circuit.h(a)
circuit.barrier(a, b, c, d)
```

# Problemsolver\_paper\_figures notebook



# NISQ vs FTQC

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import QFT
from collections import Counter
import pandas as pd

clifford_set = {"h", "cx", "s", "sdg", "x", "z"}
rotation_set = {"rz", "ry", "rx"}
t_set = {"t", "tdg"}

def classify_gates(circuit):
    ops = circuit.count_ops()
    counts = Counter()
    for g, c in ops.items():
        if g in clifford_set:
            counts["Clifford"] += c
        elif g in rotation_set:
            counts["Rotation"] += c
        elif g in t_set:
            counts["T"] += c
        else:
            counts["Other"] += c
    return counts

def ftqc_estimate(counts, logical_qubits=3):
    # Assume each logical qubit = 500 physical qubits
    base_phys = logical_qubits * 500

    # Each rotation  $\approx$  3 T-gates
    total_T = counts["T"] + counts["Rotation"] * 3

    # Physical qubits = base + 4 per T-gate (simplified assumption)
    phys_qubits = base_phys + total_T * 4

    # Runtime  $\sim$  proportional to T count
    runtime_score = total_T

    return {"physical_qubits": phys_qubits, "runtime_score": runtime_score, "total_T": total_T}
```

```
def ftqc_estimate(counts, logical_qubits=3):
    # Assume each logical qubit = 500 physical qubits
    base_phys = logical_qubits * 500

    # Each rotation  $\approx$  3 T-gates
    total_T = counts["T"] + counts["Rotation"] * 3

    # Physical qubits = base + 4 per T-gate (simplified assumption)
    phys_qubits = base_phys + total_T * 4

    # Runtime  $\sim$  proportional to T count
    runtime_score = total_T

    return {"physical_qubits": phys_qubits, "runtime_score": runtime_score, "total_T": total_T}

# --- Example Circuit: 3-qubit QFT ---
qc = QFT(3).decompose()
counts = classify_gates(qc)

# NISQ metrics
nisq_total_gates = sum(counts.values())

# FTQC metrics
ftqc = ftqc_estimate(counts, logical_qubits=3)

# Create a comparison table
data = {
    "Metric": ["Total Gates (NISQ)", "Clifford Gates", "Rotation Gates", "T-gates",
               "Physical Qubits (FTQC)", "Runtime Score (FTQC)", "Total T-equiv (FTQC)"],
    "Value": [nisq_total_gates, counts["Clifford"], counts["Rotation"], counts["T"],
              ftqc["physical_qubits"], ftqc["runtime_score"], ftqc["total_T"]]
}

df = pd.DataFrame(data)

print("QFT Circuit Comparison: NISQ vs FTQC")
```



# NISQ vs FTQC

```
print(qft_circuit_comparison_nisq_vs_ftqc)
print(df)
```

```
# Draw the circuit
qc.draw("mpl")
```

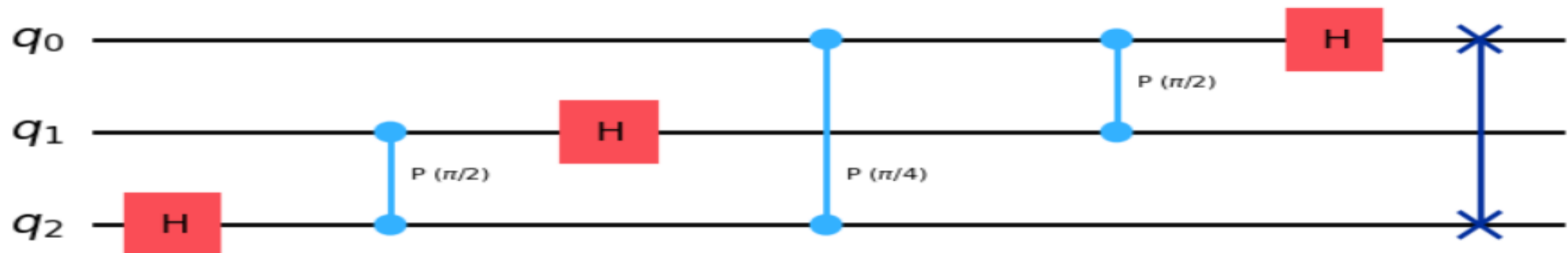
✓ [1] 4s 145ms

C:\Users\PC\AppData\Local\Temp\ipykernel\_13260\2568608932.py:42: DeprecationWarning: The class ``qiskit.circuit.library.QFTGate`` or ``qiskit.synthesis.qft.synth\_qft\_full`` instead, for access to all previous arguments.

```
qc = QFT(3).decompose()
```

QFT Circuit Comparison: NISQ vs FTQC

|   | Metric                 | Value |
|---|------------------------|-------|
| 0 | Total Gates (NISQ)     | 7     |
| 1 | Clifford Gates         | 3     |
| 2 | Rotation Gates         | 0     |
| 3 | T-gates                | 0     |
| 4 | Physical Qubits (FTQC) | 1500  |
| 5 | Runtime Score (FTQC)   | 0     |
| 6 | Total T-equiv (FTQC)   | 0     |



NEWS!

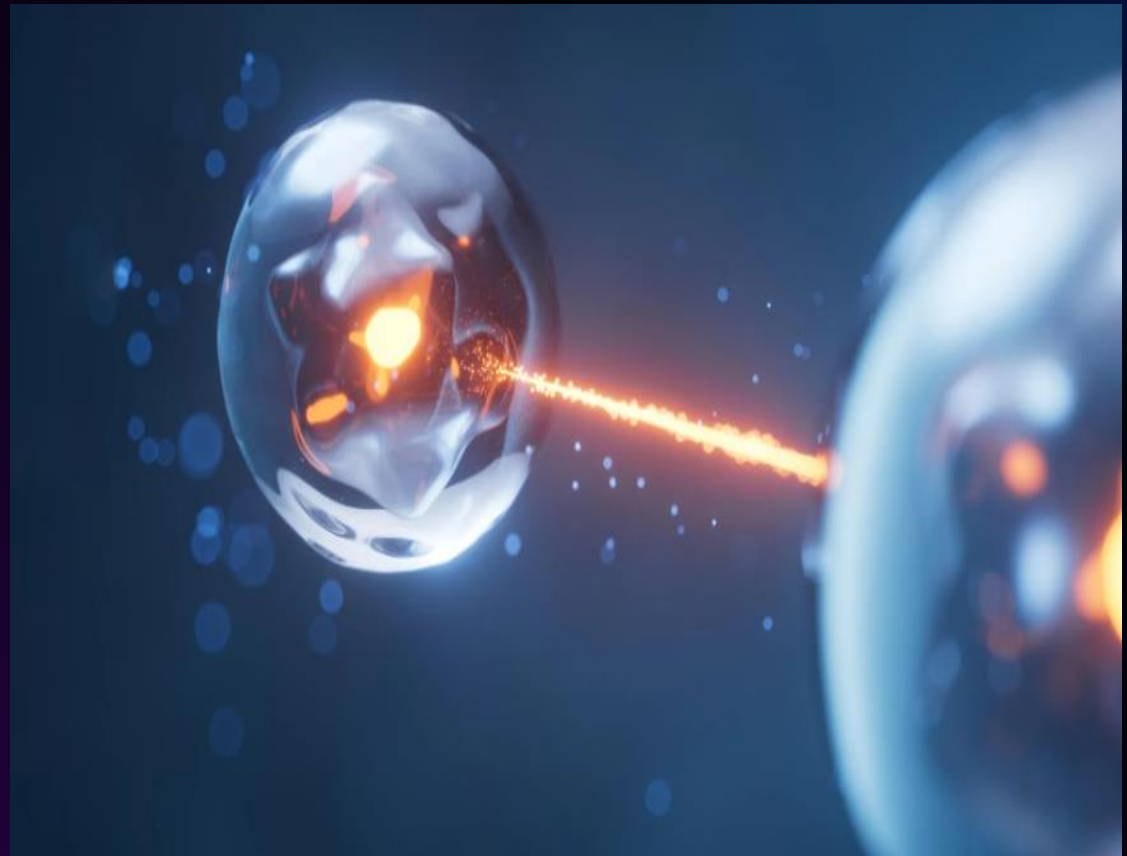
# STRANGE “HEAVY” ELECTRONS COULD BE THE FUTURE OF QUANTUM COMPUTING

[Strange “heavy” electrons could be the future of quantum computing | ScienceDaily](#)

## DETAILS:

---

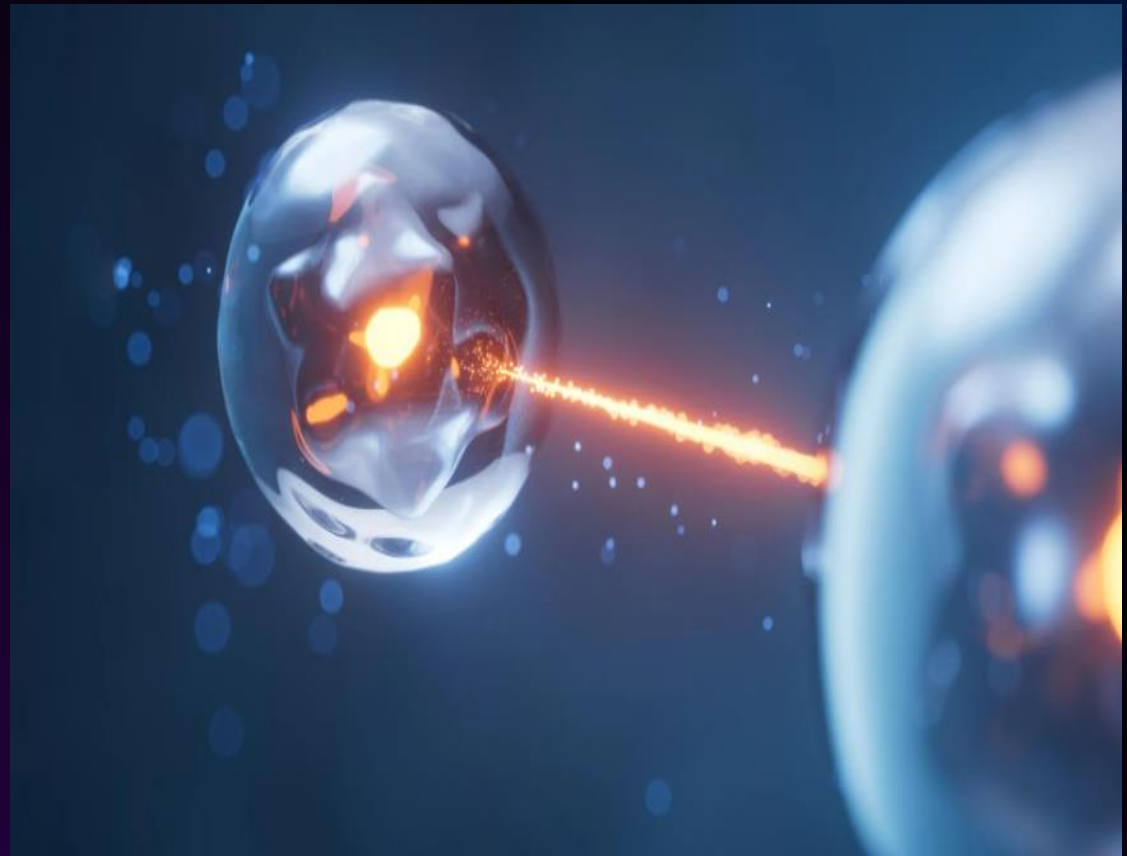
- Scientists in Japan have uncovered a strange new behavior in “heavy” electrons — particles that act as if they carry far more mass than usual. These electrons were found to be entangled, sharing a deep quantum link, and doing so in ways tied to the fastest possible time in physics. Even more surprising, the effect appeared close to room temperature, hinting that future quantum computers might harness this bizarre state of matter.



## DETAILS:

---

- Dr. Shin-ichi Kimura of The University of Osaka, who led the research, explains, "Our findings demonstrate that heavy fermions in this quantum critical state are indeed entangled, and this entanglement is controlled by the Planckian time. The Planckian time limit observed in this study provides crucial information for designing such systems. This direct observation is a significant step towards understanding the complex interplay between quantum entanglement and heavy fermion behavior."



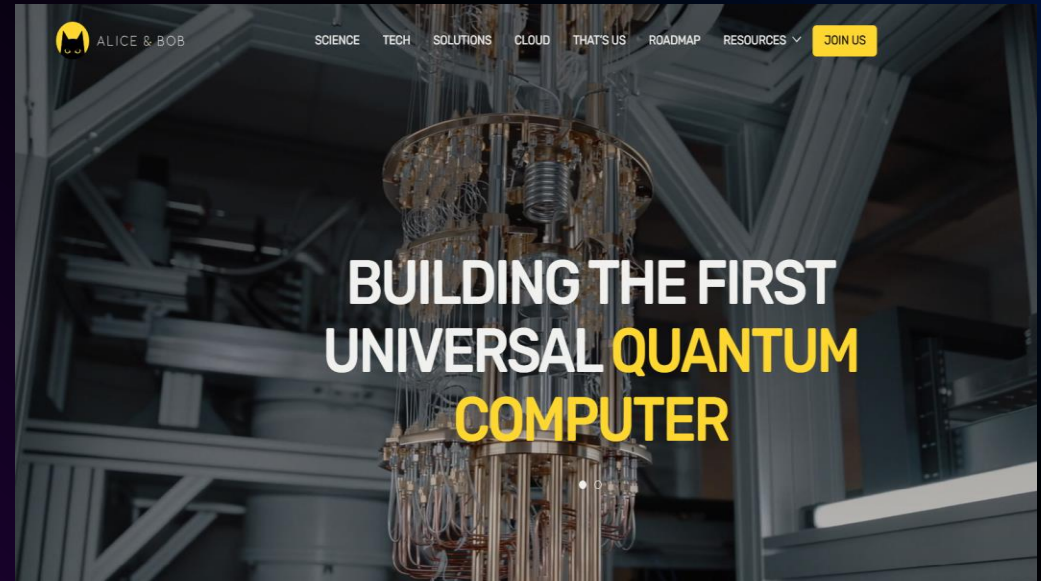
COMPANY  
ALICE  
& BOB

<https://alice-bob.com/>

# ALICE & BOB

---

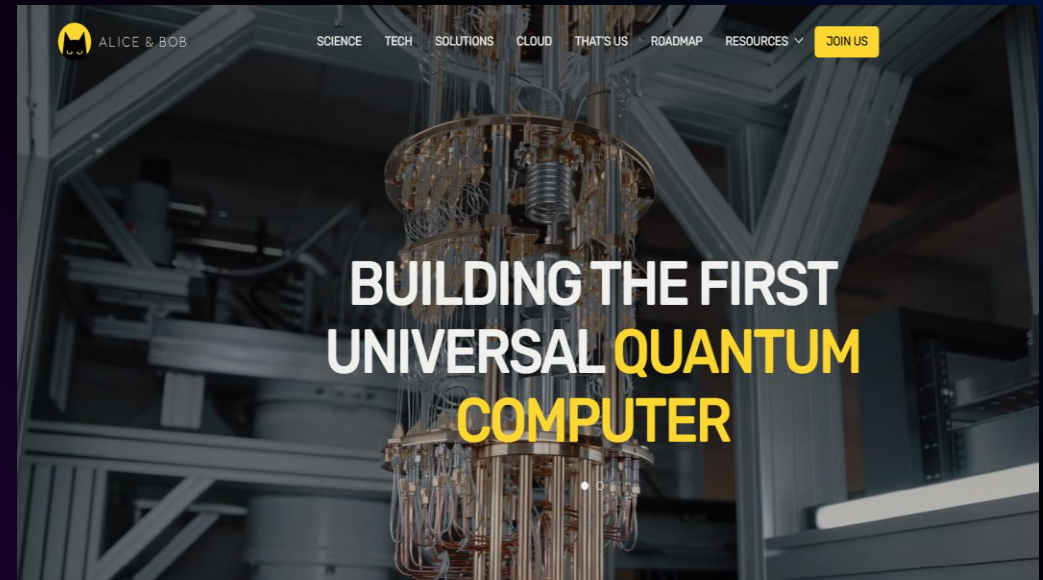
- Alice&Bob is developing a universal fault tolerant quantum computer based upon research developed at French universities and research institutes. The technology uses superconducting detection of microwave photons and what are known as "cat" qubits to exponentially reduce bit-flip errors rates. The company was founded in February 2020 and is located in Paris, France.





# ALICE & BOB

- Alice & Bob use **cat qubits**, a special type of superconducting qubits inspired by Schrödinger's cat. This technology is unique because it naturally protects against **bit-flip errors** at the hardware level, making the qubits more stable. The remaining errors, such as phase flips, are corrected using advanced **quantum error correction** techniques. By combining intrinsic error resistance with software correction, Alice & Bob can build reliable **logical qubits** with far fewer physical qubits, paving the way toward a **fault-tolerant quantum computer by 2030**.



# THANK YOU

---

Made by: Jana Mamdouh Hassan  
Moustafa hegazy

Alamein International University